DAY 5 – LAB 5.3

Contoso Orders – End-to-End Serverless Order Processing

1. Business Scenario

Contoso Orders has deployed an ASP.NET Core order application to Azure App Service.

Customers can submit orders, and the application publishes each order to the existing `orders` queue in Azure Queue Storage using Managed Identity.

The remaining requirement is to process these orders asynchronously.

You are responsible for implementing an Azure Function that automatically monitors the queue, receives submitted messages, validates order details and records successful processing.

Invalid messages must be handled through retry and poison-message handling rather than being silently discarded.

Business objective: Implement an event-driven order-processing workflow using Azure Functions and Azure Queue Storage.

What we will accomplish:

- Build an Azure Function using .NET 10 isolated worker.
- Configure a Queue Storage trigger.
- Authenticate the Function using Managed Identity.
- Configure identity-based Storage connections.

- Deploy the Function App to Azure.
- Process orders submitted through the existing Web App.
- Monitor execution using Application Insights.
- Demonstrate retry and poison-message handling.

2. Architecture & Technology Overview

End-to-end architecture

None

```
            Customer
                |
                v
ASP.NET Core Web App
  Azure App Service
                |
                | 1. Submit order
                v
  Managed Identity
                |
                | 2. Publish message
                v
Azure Storage Account
                |
                v
   Queue: orders
                |
                | 3. Queue Trigger
                v
 Azure Functions
   .NET 10 Isolated
```



How the components work together

1. Customer submits an order through the Web App.
2. ASP.NET Core publishes a Base64-encoded JSON message to the `orders` queue.
3. Azure Functions detects the new queue message and starts execution.
4. The Function validates and processes the order.
5. Successful execution is recorded in application logs.
6. Failed messages are retried and eventually moved to the poison queue after the configured maximum attempts.

Key Concepts

Component	Purpose

Azure Functions	Runs code in response to events without requiring us to manage application servers.
Function App	Provides the hosting and management boundary for one or more functions.
.NET Isolated Worker	Runs application code in a process separate from the Functions host.
Queue Trigger	Automatically invokes a Function when a new queue message is available.
Azure Queue Storage	Holds messages until they are successfully processed or moved to the poison queue.
Managed Identity	Authenticates the Function without storing Storage credentials.
`host.json`	Defines Function-wide runtime and trigger configuration.
Application Insights	Provides execution telemetry, logs and failure investigation.
Poison Queue	Holds messages that repeatedly fail processing.

Understanding Azure Functions

Azure Functions is a serverless compute service used to execute code in response to events.

Unlike a continuously running application, a queue-triggered Function executes when the configured queue receives a message.

Real-world use cases:

- Order processing.
- Sending customer notifications.

- Processing uploaded documents.
- Generating reports.
- Background data transformation.
- Event-driven application integration.

Why are we using Azure Functions?

Our Web App should not be responsible for completing every downstream order-processing activity.

Instead, it publishes the order to a queue and allows a separate processing component to handle it.

This improves separation of responsibilities and allows the processing component to scale independently.

Understanding the Isolated Worker Model

In the isolated worker model, the Function application runs in a separate .NET worker process from the Azure Functions host.

This provides:

- Independent application dependency management.
- Standard .NET dependency injection.
- Greater control over application startup.
- Flexibility to target supported .NET versions.

We are using the isolated worker model with .NET 10.

Understanding Queue Triggers

A Queue Trigger automatically invokes a Function when a message becomes available in the configured queue.

The developer does not need to continuously poll the queue manually.

The Functions runtime manages message retrieval, execution, retries and poison-message handling.

Understanding Retry and Poison Messages

When a Function fails while processing a queue message, the message becomes available again after the configured visibility timeout.

The Functions runtime tracks dequeue attempts.

After the configured maximum number of attempts, the message is moved to a separate poison queue for investigation.

This prevents repeatedly failing messages from blocking normal processing.

Hosting Decision – Flex Consumption

We will use the Flex Consumption hosting plan.

.NET 10 isolated worker is supported on Functions runtime v4. For Linux hosting, .NET 10 requires Flex Consumption rather than the classic Linux Consumption plan.

Flex Consumption provides event-driven scaling and supports Managed Identity for Function host storage.

This is a deliberate runtime and hosting compatibility decision.

3. Resources and Naming

Reuse the infrastructure created in Labs 5.1 and 5.2.

Resource	Example name	Action
Resource Group	Existing shared RG	Reuse

Storage Account	stuser01ordersd5	Reuse
Queue	orders	Reuse
App Service	web-user01-orders-d5	Reuse
Function App	fn-user01-orderproc-d5	Create
Function Project	ContosoOrders.Processor	Create
Application Insights	appi-user01-orderproc-d5	Create or reuse the instance created with the Function App

Replace `user01` with the assigned participant suffix.

Hosting configuration:

- Functions runtime: Version 4.
- Runtime stack: .NET 10 isolated.
- Operating System: Linux.
- Hosting Plan: Flex Consumption.

Important: Confirm that the selected Azure region supports Flex Consumption.

4. Prerequisites

- Labs 5.1 and 5.2 completed.
- Existing `orders` queue available.
- .NET 10 SDK installed.
- Azure Functions Core Tools v4 with .NET 10 template support.
- VS Code.

- Azure Functions extension for VS Code.
- Azure subscription access.
- Permission to create Function Apps and Azure role assignments.
- Azurite for optional local queue-trigger testing.
- Access to Application Insights.

---

5. Lab Execution

Phase 1 – Create the .NET 10 Function Project

Concept – What are we building?

We are creating a .NET 10 isolated Azure Functions project that will contain the background order-processing logic.

The project is separate from the ASP.NET Core Web App.

Why are we creating a separate project?

The Web App handles customer requests, while the Function handles background order processing.

Keeping these responsibilities separate makes the application easier to maintain and scale.

Execution

Open VS Code Terminal.

Create the project directory:

```
None
mkdir ContosoOrders.Processor
```

```
cd ContosoOrders.Processor
```

Initialize the Function project:

```
None
func init . --worker-runtime dotnet-isolated
--target-framework net10.0
```

If the installed Azure Functions Core Tools version does not offer the .NET 10 template, update Core Tools before proceeding.

Do not generate a .NET 8 project and simply change its target framework.

Validate the project dependencies

Execute:

```
None
dotnet --version
func --version
```

Inspect the generated project file:

```
None
ContosoOrders.Processor.csproj
```

For .NET 10, verify that:

- Target framework is `net10.0`.
- Microsoft.Azure.Functions.Worker is version 2.50.0 or later.

- The Functions build SDK is Azure.Functions.Sdk 1.0.0 or later, or the project uses an equivalent supported and validated SDK configuration.

The current Microsoft guidance identifies these minimum versions for .NET 10 isolated projects.

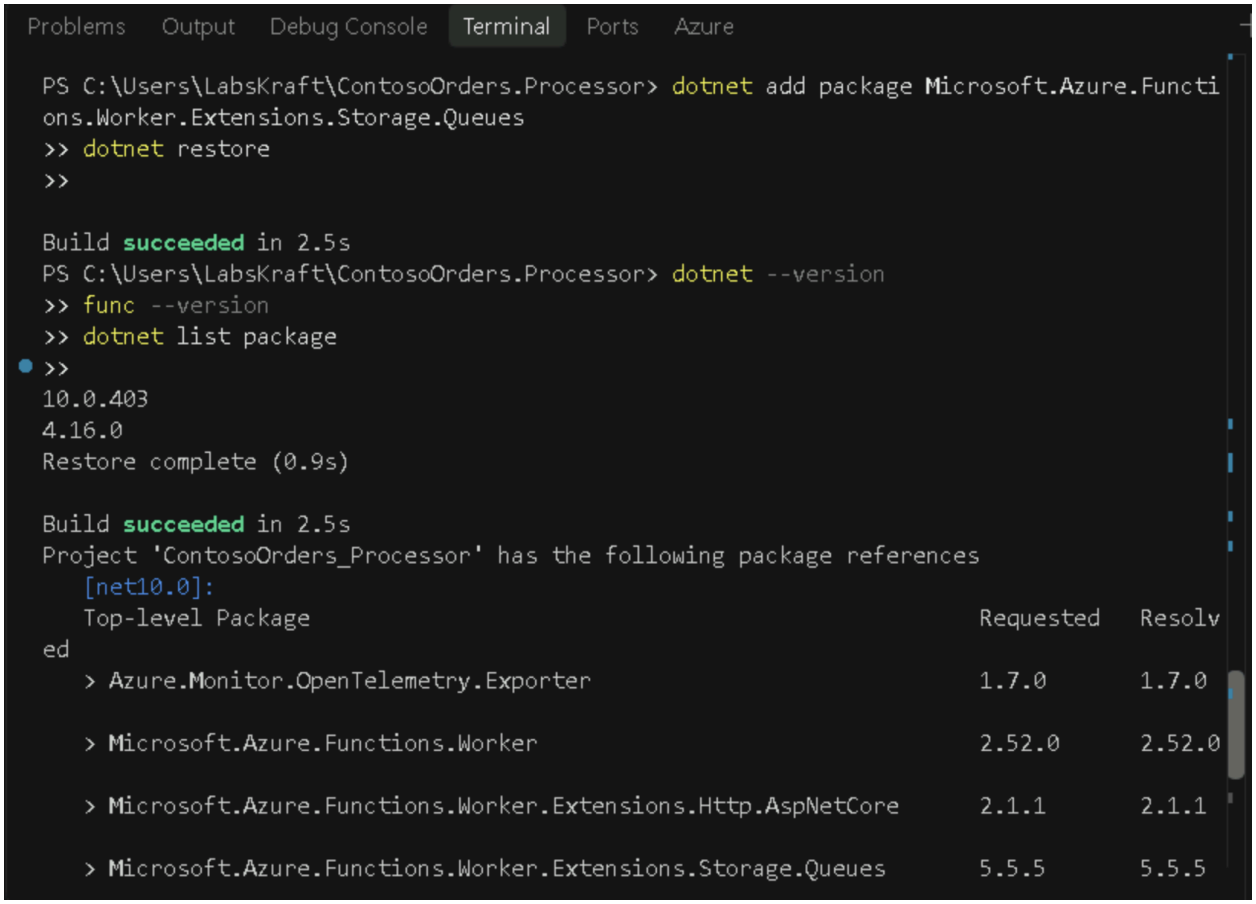
Add the Queue Storage extension

Execute:

```
None
dotnet add package
Microsoft.Azure.Functions.Worker.Extensions.Storage.Q
ueues
dotnet restore
```

Validate:

```
None
dotnet list package
```



Confirm that the Queue Storage extension is version 5.x or later, which supports identity-based connections and the required Base64 message configuration.

Expected outcome: The Function project targets .NET 10 isolated worker and includes the required Queue Storage extension.

Phase 2 – Implement the Queue Processor

Concept – How will the Function execute?

We will use a Queue Trigger.

Whenever a new message arrives in the `orders` queue, Azure Functions invokes the associated Function method.

The Function will deserialize the message, validate its contents and log the processing result.

Execution

Create:

`ProcessOrder.cs`

Add:

```
None
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;
using System.Text.Json;

namespace ContosoOrders.Processor;

public class ProcessOrder
{
    private readonly ILogger<ProcessOrder> _logger;

    public ProcessOrder(ILogger<ProcessOrder> logger)
    {
        _logger = logger;
    }

    [Function(nameof(ProcessOrder))]
    public void Run(
        [QueueTrigger("orders", Connection =
"OrdersQueue")]
        string message)
```

```
    {
        _logger.LogInformation("Order message
received.");

        try
        {
            var order =
JsonSerializer.Deserialize<OrderMessage>(
                message,
                new JsonSerializerOptions
                {
                    PropertyNameCaseInsensitive =
true
                });

            if (order is null ||

string.IsNullOrWhiteSpace(order.OrderId) ||

string.IsNullOrWhiteSpace(order.CustomerName) ||

string.IsNullOrWhiteSpace(order.Product) ||
                order.Quantity < 1)
            {
                throw new InvalidOperationException(
                    "Order message failed
validation.");
            }
```

```
            _logger.LogInformation(
                "Order processed. OrderId={OrderId},
Customer={Customer}, Product={Product},
Quantity={Quantity}",
                order.OrderId,
                order.CustomerName,
                order.Product,
                order.Quantity);
        }
        catch (JsonException ex)
        {
            _logger.LogError(ex, "Invalid order
JSON.");
            throw;
        }
        catch (InvalidOperationException ex)
        {
            _logger.LogError(ex, "Order validation
failed.");
            throw;
        }
    }
}

public class OrderMessage
{
    public string? OrderId { get; set; }
    public string? CustomerName { get; set; }
    public string? Product { get; set; }
```

```
    public int Quantity { get; set; }
    public DateTime SubmittedAtUtc { get; set; }
}
```

Understand the implementation

Code component	Responsibility
`[Function]`	Registers the method as an Azure Function.
`[QueueTrigger]`	Connects the Function to the `orders` queue.
`ILogger`	Records processing and failure information.
`JsonSerializer`	Converts JSON message content into a C# object.
`OrderMessage`	Defines the expected order structure.
Validation conditions	Prevent invalid order details from being processed.
`throw`	Reports failure to the Functions runtime so that the message can be retried.

Behind the scenes

1. The Functions host detects an available queue message.
2. It retrieves the message and invokes the Function.
3. The isolated worker executes the C# processing code.
4. The message is deserialized into an OrderMessage object.
5. Validation determines whether the order is acceptable.
6. Successful execution allows the message to be deleted from the queue.

7. An exception signals unsuccessful processing, allowing the retry mechanism to operate.

Important: Do not remove the `throw` statements from the exception handlers. If the Function catches an error without rethrowing it, the runtime may treat the invocation as successful and remove the message.

Expected outcome: The Function contains the queue-triggered order-processing logic and validates the expected message structure.

Phase 3 – Configure Retry and Poison-Message Handling

Concept – Why do we need retry handling?

In production, messages may fail because of invalid data, temporary service unavailability or application errors.

Retry handling allows the system to attempt processing again instead of immediately losing the message.

Messages that continue to fail are moved to a poison queue for investigation.

Execution

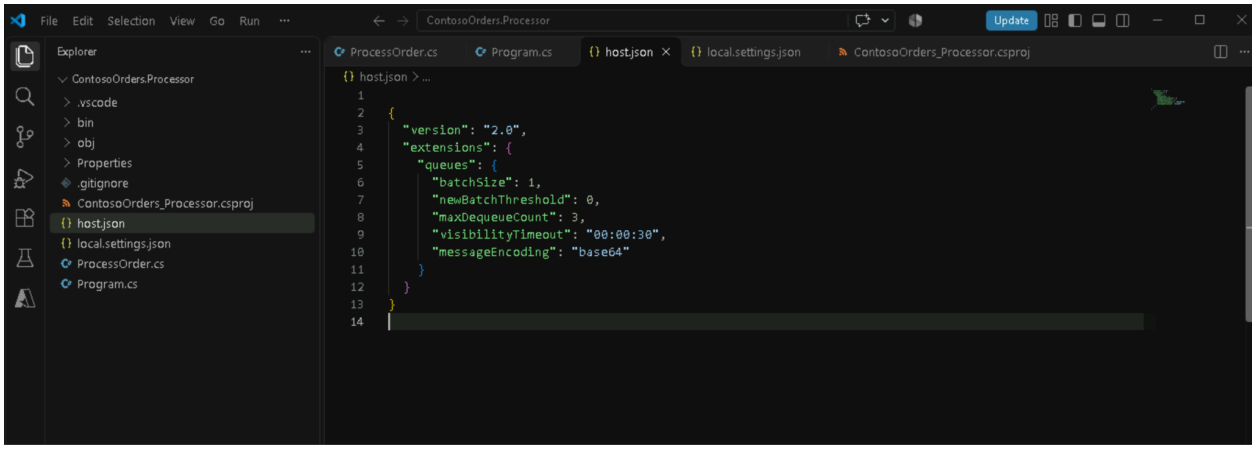
Create or update:

`host.json`

Add:

```
None
{
  "version": "2.0",
  "extensions": {
    "queues": {
```

```
        "batchSize": 1,
        "newBatchThreshold": 0,
        "maxDequeueCount": 3,
        "visibilityTimeout": "00:00:30",
        "messageEncoding": "base64"
    }
  }
}
```



Understand the configuration

Setting	Value	Purpose
batchSize	1	Processes one message at a time per Function host instance.
newBatchThreshold	0	Prevents the runtime from prefetching another batch at the configured threshold.
maxDequeueCount	3	Allows up to three processing attempts before poison handling.
visibilityTimeout	30 seconds	Makes a failed message visible again after the configured interval.

messageEncoding	base64	Decodes Base64-encoded queue messages.

Important: `batchSize = 1` does not guarantee globally sequential processing if Flex Consumption scales the Function App to multiple instances.

Poison queue understanding

If the `orders` message repeatedly fails, Azure Functions moves it to:

```
None
orders-poison
```

The poison queue is created by the Queue Trigger runtime when needed.

Expected outcome: The Function has a defined retry policy and poison-message destination.

Phase 4 – Configure and Validate the Function Locally

Concept – Why validate locally?

Local validation helps identify project, dependency, startup and trigger configuration issues before deploying to Azure.

For this lab, we will use Azurite to simulate Azure Storage locally.

Step 1 – Configure Program.cs

Create or update:

`Program.cs`

Add:

```
None
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Hosting;

var host = new HostBuilder()
    .ConfigureFunctionsWorkerDefaults()
    .Build();

host.Run();
```

This initializes the isolated worker host and starts the Functions application.

Step 2 – Configure local.settings.json

Create:

`local.settings.json`

Add:

```
None
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated",
    "OrdersQueue": "UseDevelopmentStorage=true"
  }
}
```

Understanding the settings

Setting	Purpose
AzureWebJobsStorage	Storage connection required by the Functions host.
FUNCTIONS_WORKER_RUNTIME	Identifies the isolated .NET worker.
OrdersQueue	Local connection used by the Queue Trigger.

These are local development settings. Azure production connections will use Managed Identity.

Do not commit `local.settings.json` containing credentials to source control.

Step 3 – Start Azurite

Start the installed Azurite service or VS Code Azurite extension.

Ensure its Blob and Queue endpoints are available.

Using Azure Storage Explorer or the Azurite interface:

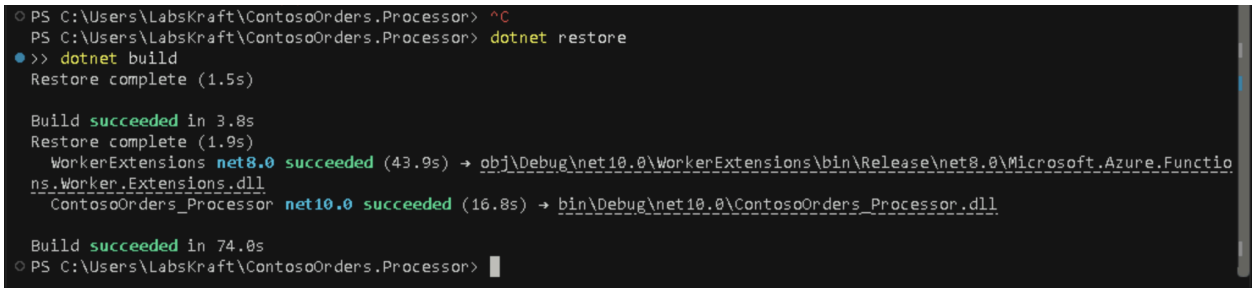
1. Connect to the local development storage.
2. Open Queue Storage.
3. Create a queue named `orders`.

Step 4 – Build the project

Execute:

```
None
dotnet restore
```

```
dotnet build
```



Expected result: The project builds successfully without errors.

Step 5 – Start the Function locally

Execute:

```
None
func start
```

Verify:

- The Functions host starts.
- The isolated worker starts.
- `ProcessOrder` is discovered.
- The `orders` Queue Trigger initializes.

Local validation note: The full local trigger test requires the `orders` queue in Azurite and a valid test message. If Azurite is not available, complete the build validation locally and perform the end-to-end trigger test after Azure deployment.

Expected outcome: The Function starts successfully and is ready to listen for queue messages.

Phase 5 – Create the Azure Function App

Concept – What is a Function App?

A Function App is the Azure hosting and management resource that provides an execution environment for one or more Azure Functions.

It manages runtime configuration, application settings, monitoring and scaling.

Why Flex Consumption?

We are targeting .NET 10 on Linux. Flex Consumption is the appropriate hosting plan for this implementation.

Execution

Open Azure Portal.

1. Search for Function App.
2. Select Create.
3. Choose Flex Consumption as the hosting option.
4. Configure:

Setting	Value
Subscription	Training subscription
Resource Group	Existing shared RG
Function App Name	fn-user01-orderproc-d5
Deployment Model	Code
Runtime Stack	.NET 10 isolated
Operating System	Linux

Hosting Plan	Flex Consumption
Region	Same as Storage Account

Storage configuration

On the Storage page, select the existing Storage Account:

`stuser01ordersd5`

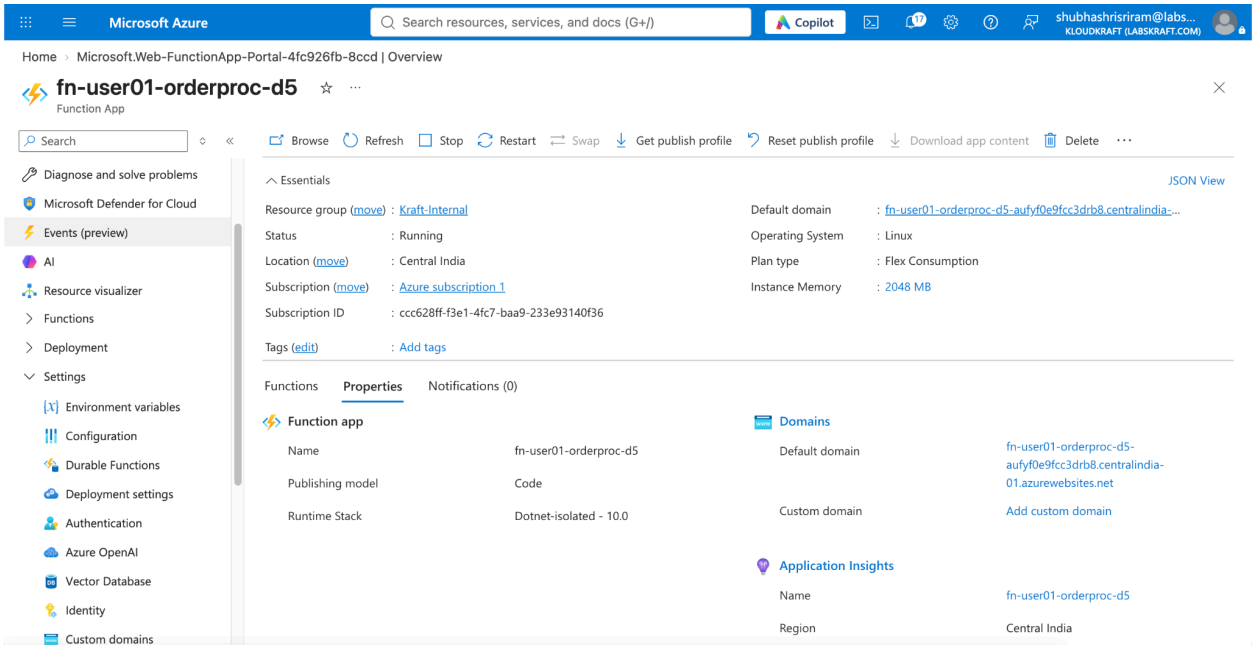
This is the account containing our `orders` queue and the storage used by the Function host in this training implementation.

Monitoring configuration

Enable Application Insights.

Create or select the available Application Insights resource.

Select Review + create → Create.



Verify the Function App

After provisioning, open the Function App and confirm:

- Runtime stack: .NET 10 isolated.
- Functions runtime: Version 4.
- Hosting plan: Flex Consumption.
- Operating system: Linux.
- Application Insights: Configured.

Expected outcome: Function App is provisioned with the required .NET 10 isolated hosting environment.

Phase 6 – Enable Function App Managed Identity

Concept – Why does the Function need its own identity?

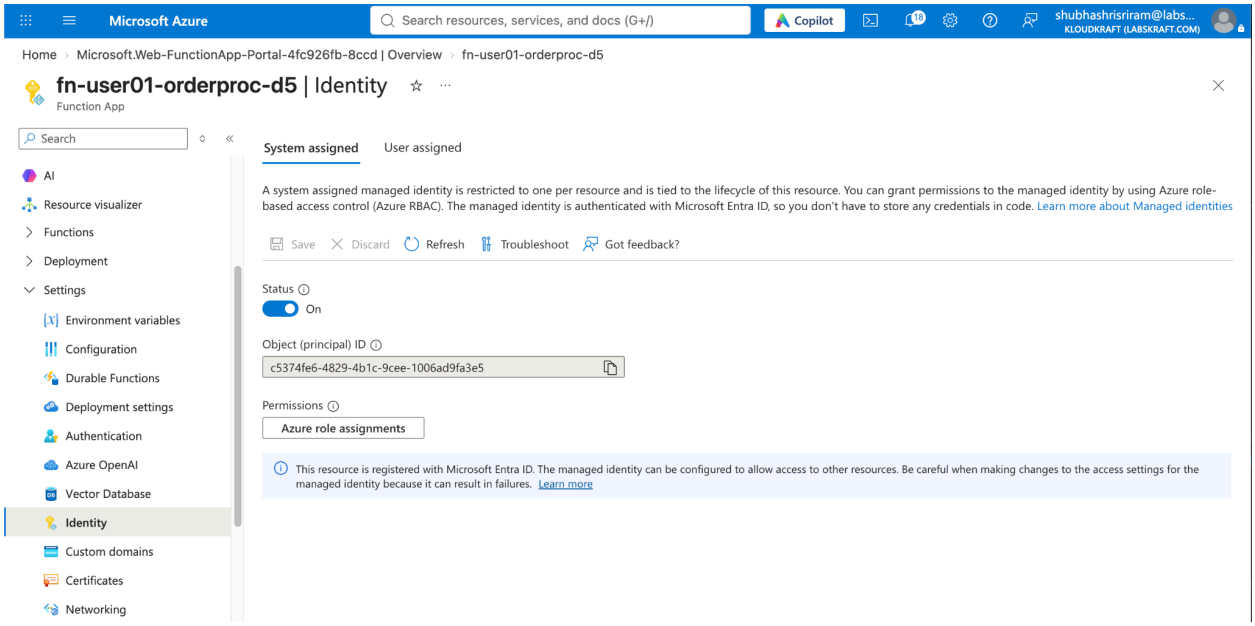
The Web App and Function App are separate Azure resources.

The Web App identity is responsible for publishing messages. The Function identity is responsible for consuming and processing them.

They must have independent authentication and authorization.

Execution

1. Open `fn-user01-orderproc-d5`.
2. Navigate to Settings → Identity.
3. Under System assigned, change Status to On.
4. Select Save.
5. Record the Object (principal) ID.



Expected outcome: Function App has its own system-assigned Managed Identity.

Phase 7 – Grant Storage Permissions to the Function Identity

Concept – Why are we assigning two roles?

The Function App uses the same Storage Account for two separate purposes:

- Function host storage operations.
- Queue Trigger message processing.

Each operation requires appropriate data-plane permissions.

Execution

Open the Storage Account:

`stuser01ordersd5`

Navigate to Access control (IAM) → Add → Add role assignment.

Role 1 – Queue processing

Assign:

`Storage Queue Data Contributor`

Select the system-assigned identity for:

`fn-user01-orderproc-d5`

Select Review + assign.

Why this role?

The Queue Trigger must retrieve messages, manage their visibility, delete successfully processed messages and write failed messages to the poison queue.

Storage Queue Data Contributor provides the required queue data permissions for this training configuration.

Role 2 – Function host storage

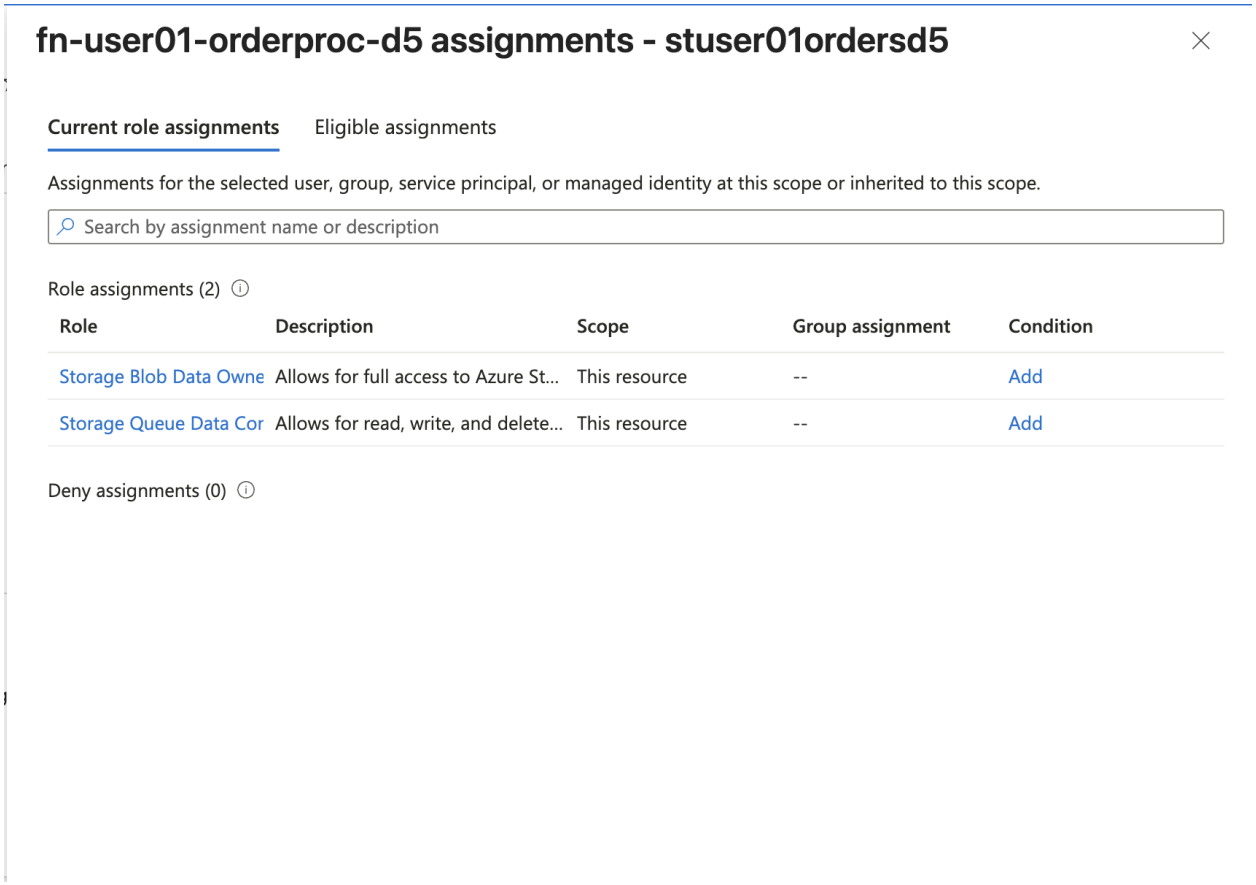
Create another role assignment.

Assign:

`Storage Blob Data Owner`

Select the same Function App identity.

Select Review + assign.



Why this role?

The Functions host uses AzureWebJobsStorage for internal operations. Blob Data Owner provides the minimum documented permissions for the standard host-storage connection.

Important distinction

Identity	Permission
Web App identity	Storage Queue Data Message Sender
Function App identity	Storage Queue Data Contributor
Function App identity	Storage Blob Data Owner

Do not grant the Function's roles to the Web App identity.

If Functions host diagnostic events cannot be written to Table Storage, review the additional Storage Table Data Contributor requirement for those diagnostics.

Expected outcome: Function App identity has the required permissions to use Queue Storage and host storage.

Allow time for RBAC role assignments to propagate.

Phase 8 – Configure Identity-based Storage Connections

Concept – Two different storage connections

The Function App uses two named storage connections.

AzureWebJobsStorage

Used by the Functions host for its internal runtime operations.

OrdersQueue

Used by the Queue Trigger to access the business queue.

Although both connections point to the same Storage Account in this lab, they serve different purposes.

Execution

Open:

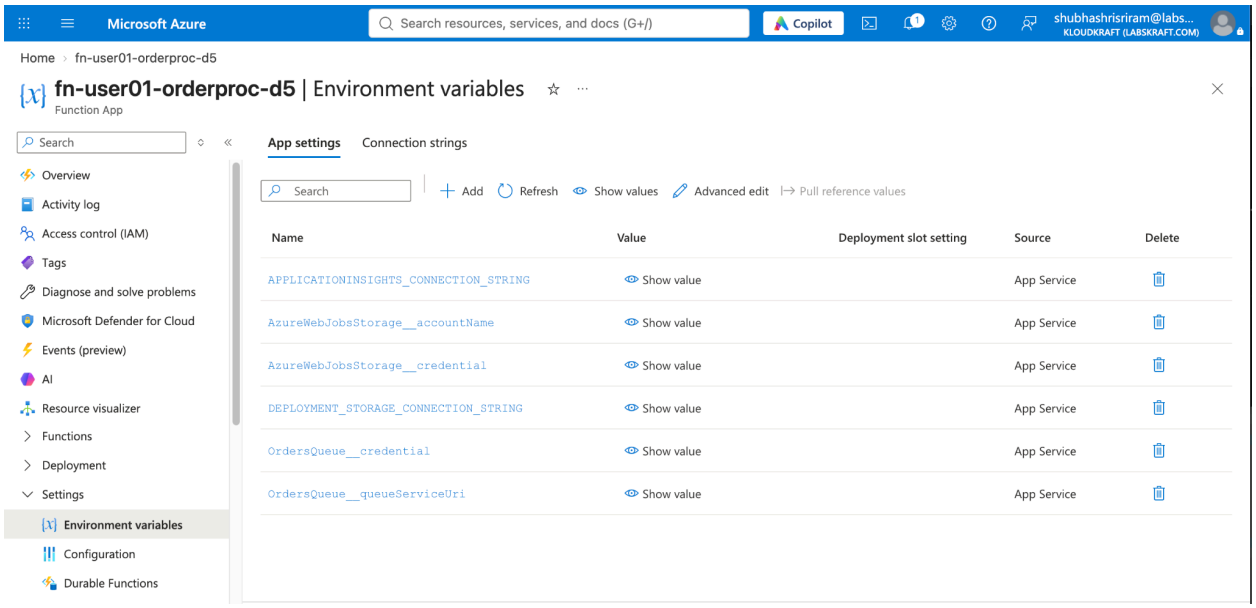
Function App → Settings → Environment variables.

Add the following application settings:

Setting name	Value
AzureWebJobsStorage__accountName	stuser01ordersd5

AzureWebJobsStorage__credential	managedidentity
OrdersQueue__queueServiceUri	https://stuser01ordersd5.queue.core.windows.net
OrdersQueue__credential	managedidentity
FUNCTIONS_WORKER_RUNTIME	dotnet-isolated
FUNCTIONS_EXTENSION_VERSION	~4

Replace the Storage Account name with your actual name.



Understand the configuration

- `AzureWebJobsStorage__accountName` identifies the host storage account.
- `AzureWebJobsStorage__credential` instructs the host to authenticate using Managed Identity.

- `OrdersQueue__queueServiceUri` identifies the queue service endpoint.
- `OrdersQueue__credential` instructs the Queue Trigger to use Managed Identity.
- `FUNCTIONS_WORKER_RUNTIME` selects the isolated worker.
- `FUNCTIONS_EXTENSION_VERSION` selects Functions runtime v4.

The prefix `OrdersQueue` matches:

```
None
[QueueTrigger("orders", Connection = "OrdersQueue")]
```

Important

If the Function App has an automatically generated `AzureWebJobsStorage` connection-string setting, remove that legacy setting only after confirming the identity-based settings are correctly configured.

Do not leave both the old connection-string form and the new identity-based form configured under the same connection name.

Also, do not delete any separate Flex Consumption deployment-storage settings such as `DEPLOYMENT_STORAGE_CONNECTION_STRING` merely because they contain a connection string. Deployment-package storage is a separate platform concern.

Save the settings.

Expected outcome: Function host storage and Queue Trigger connections are configured to use the Function App's system-assigned Managed Identity.

Phase 9 – Deploy the Function Project

Concept – Function Deployment

We are publishing the locally developed Function project to the Azure Function App.

Once deployed, the Azure Functions host discovers the Function metadata and activates the configured Queue Trigger.

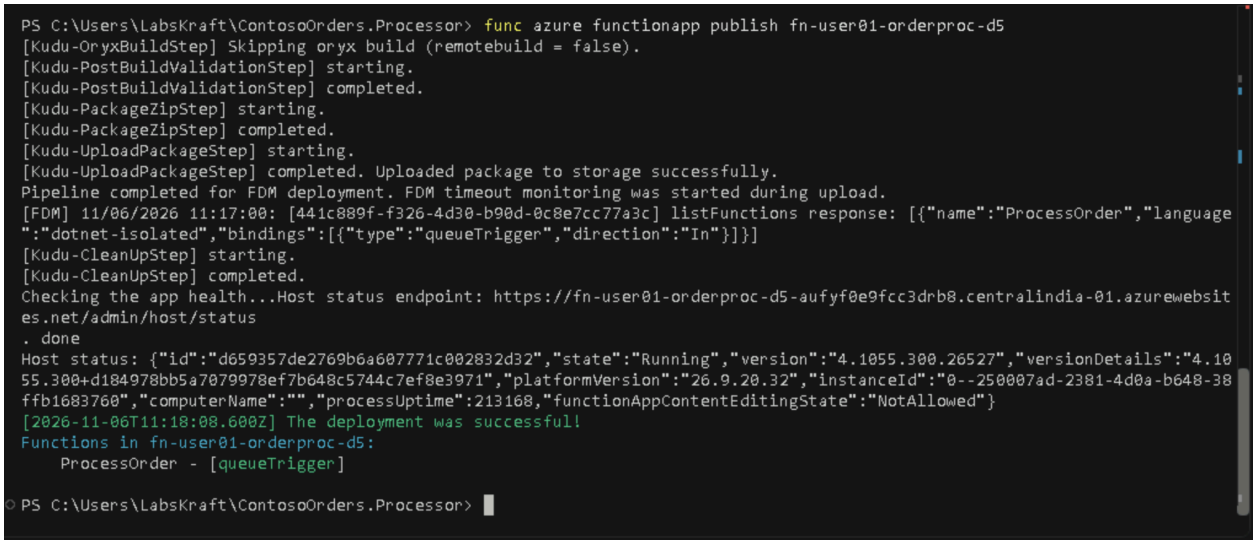
Execution using VS Code

1. Open the `ContosoOrders.Processor` folder.
2. Sign in to Azure using the Azure Functions extension.
3. Expand Workspace → Functions.
4. Select Deploy to Function App.
5. Choose `fn-user01-orderproc-d5`.
6. Confirm deployment when prompted.
7. Wait for deployment to complete.

Alternative – Core Tools deployment

If using the command line:

```
None
func azure functionapp publish fn-user01-orderproc-d5
```



After deployment:

1. Open the Function App in Azure Portal.
2. Navigate to Functions.
3. Verify that `ProcessOrder` is listed.

Expected outcome: The Function project is deployed and the `ProcessOrder` Function is visible in Azure.

Phase 10 – Verify Function Configuration

Before testing, perform the following checks.

Open:

Function App → Settings → Environment variables.

Verify:

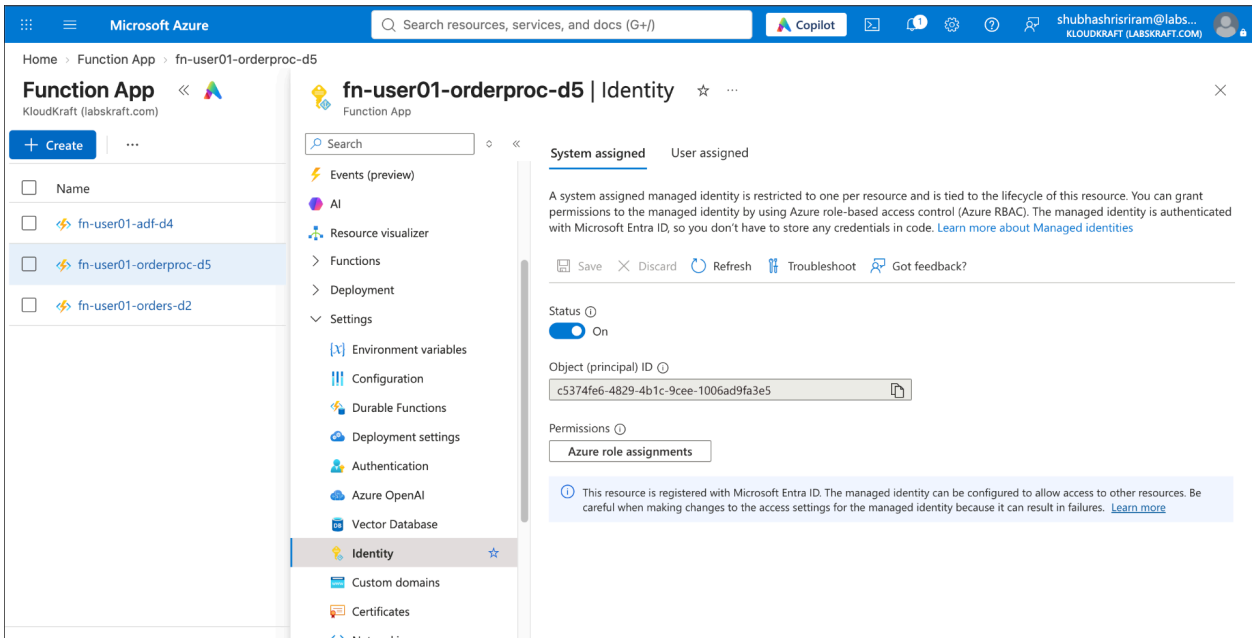
- `AzureWebJobsStorage__accountName` points to the correct account.
- `AzureWebJobsStorage__credential` is `managedidentity`.
- `OrdersQueue__queueServiceUri` points to the correct Queue Storage endpoint.

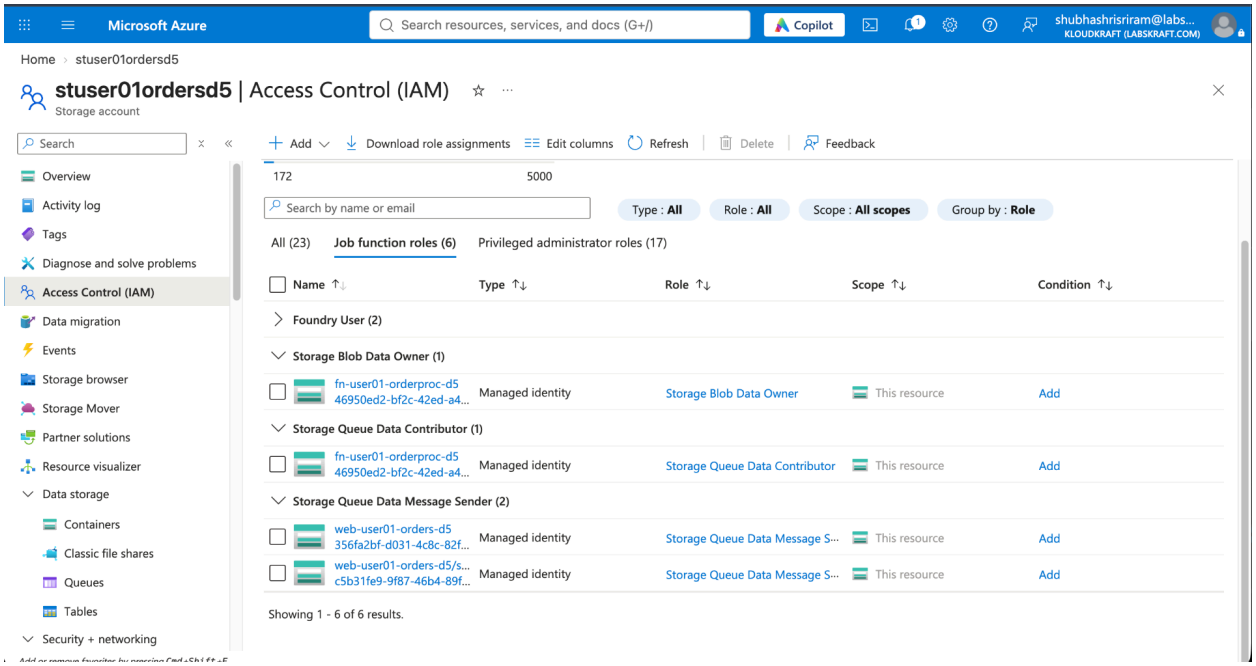
- `OrdersQueue__credential` is `managedidentity`.
- `FUNCTIONS_WORKER_RUNTIME` is `dotnet-isolated`.
- `FUNCTIONS_EXTENSION_VERSION` is `~4`.

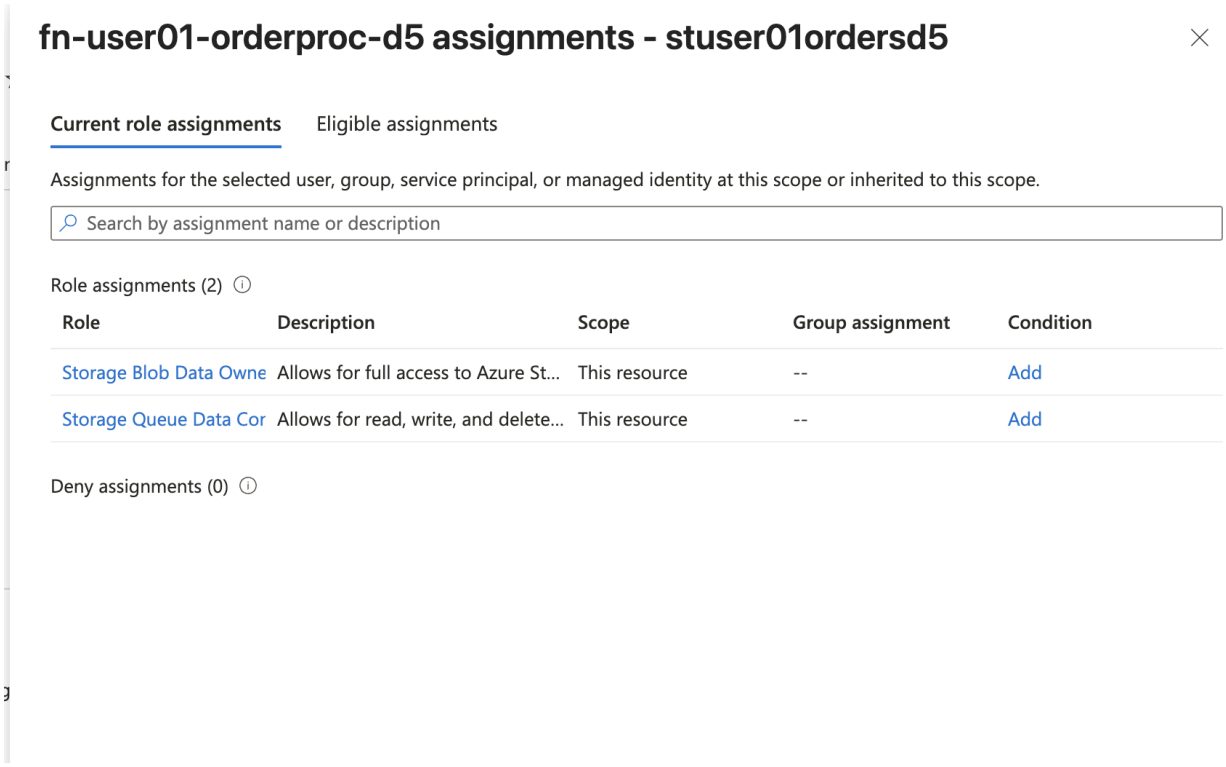
Also verify:

- System-assigned Managed Identity is enabled.
- Storage Queue Data Contributor is assigned to the Function identity.
- Storage Blob Data Owner is assigned to the Function identity.
- The `orders` queue exists.
- Application Insights is connected.

Restart the Function App after configuration changes if required.







Expected outcome: Function App runtime, identity, storage connections and permissions are ready for execution.

Phase 11 – Run the End-to-End Order Processing Test

Purpose: Validate the complete integration established across Labs 5.1, 5.2 and 5.3.

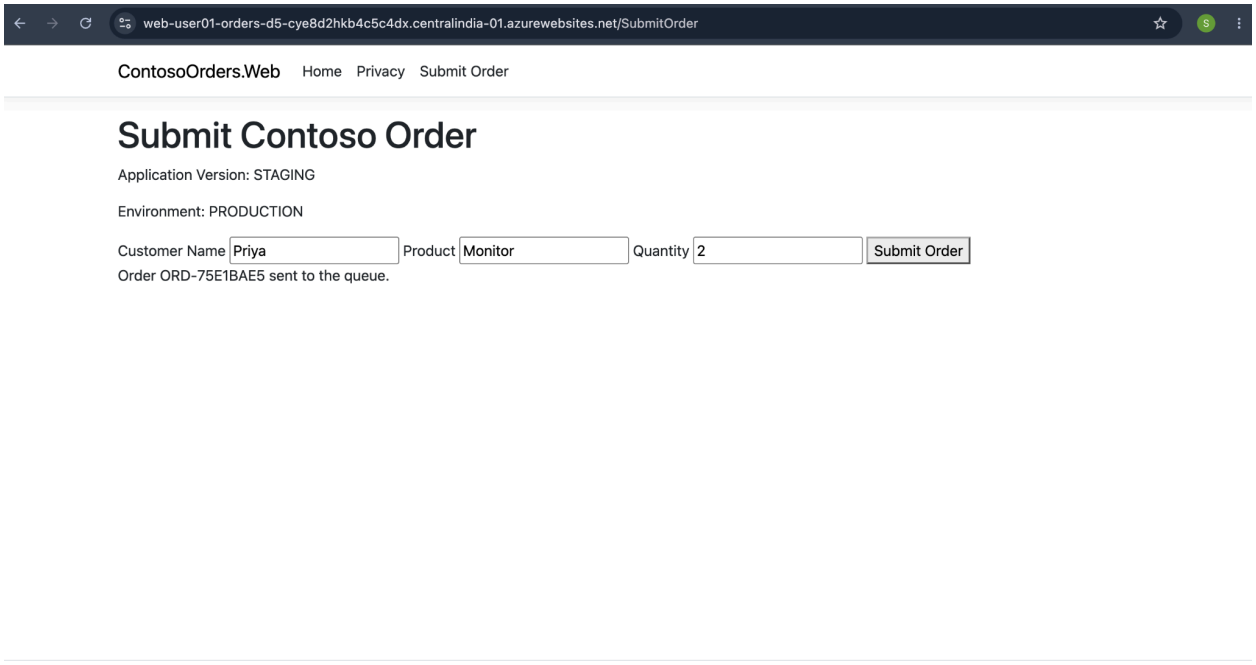
Execution

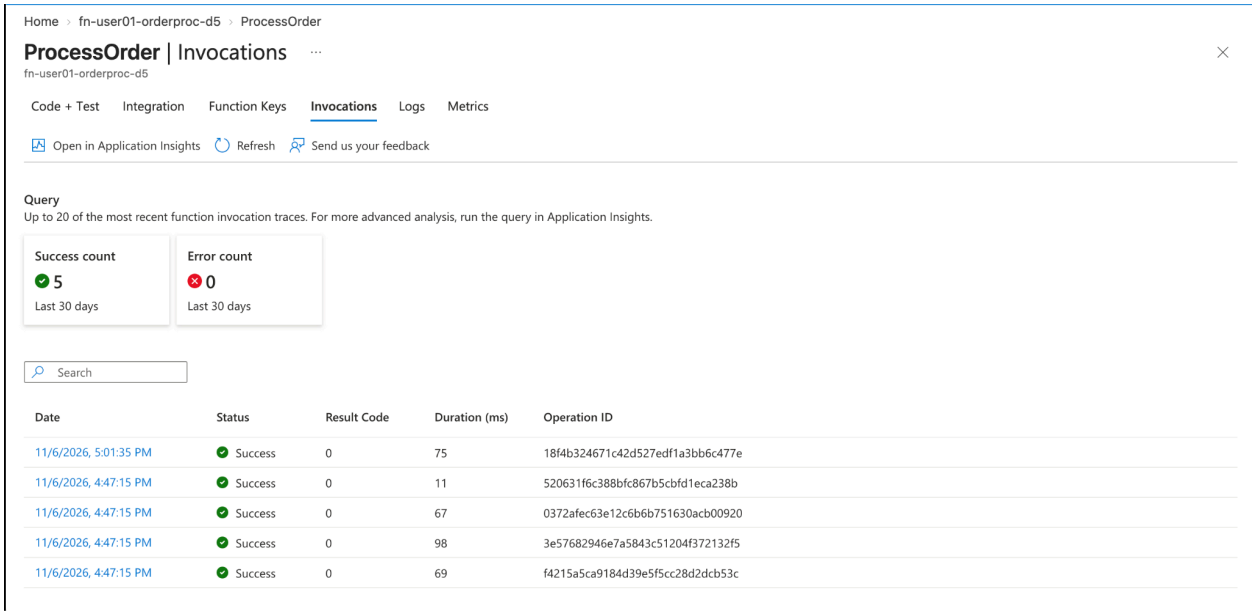
1. Open the production Web App URL.
2. Navigate to Submit Order.
3. Submit the following test order:

Field	Value
Customer Name	Priya
Product	Monitor
Quantity	2

4. Verify that the Web App confirms the order was sent.
5. Open Storage Account → Queues → `orders`.
6. Observe the queue if the message is still available.
7. Open Function App → Monitoring → Log Stream, or open Application Insights → Logs.
8. Search for the generated Order ID.

Important: The Function may process the message quickly. It may disappear from the active queue before you inspect it. This is expected when processing succeeds.





Expected result

The order is received and processed by the Function.

The logs should show:

- Generated Order ID.
- Customer name.

- Product.
- Quantity.
- Successful processing message.

Understand the complete execution flow

Step	Component	Activity
1	Web App	Accepts the order.
2	Managed Identity	Authenticates the Web App.
3	Queue Storage	Receives and stores the message.
4	Queue Trigger	Detects the message.
5	Function App	Executes the order processor.
6	Managed Identity	Authorizes Function access to Storage.
7	Application Insights	Captures execution logs and telemetry.

Expected outcome: A customer order submitted through the Web App is automatically processed by Azure Functions.

Phase 12 – Monitor Processing Using Application Insights

Concept – Why Application Insights?

In production, applications process events continuously. Operations teams need visibility into execution, failures and performance.

Application Insights provides telemetry and diagnostic information to investigate Function execution.

Execution

Open:

Azure Portal → Application Insights → appi-user01-orderproc-d5 → Logs.

Run:

```
None
traces
| where message contains "Order processed"
| order by timestamp desc
```

To inspect failures:

```
None
exceptions
| order by timestamp desc
```

To inspect recent Function traces:

```
None
traces
| where timestamp > ago(30m)
| order by timestamp desc
```

If the Application Insights resource has a different name, open the instance associated with the Function App.

Expected outcome: Successful processing logs and available exception information can be inspected.

Real-world understanding

An operations engineer can use the Order ID to trace an individual order through the application logs and investigate failures without manually accessing the application server.

Phase 13 – Test Retry and Poison-Message Handling

Concept – What happens when processing fails?

In production, some messages may contain invalid business data or malformed JSON.

The Function should not silently accept these messages.

When processing fails, the Function throws an exception. The Functions runtime retries the message according to the configured queue settings.

After three unsuccessful attempts, the message moves to the poison queue.

Execution

Step 1 – Prepare an invalid order message

Open:

Storage Account → Queues → `orders`.

Use the available queue-message insertion option or Azure Storage Explorer.

We will test a syntactically valid JSON message containing invalid business data.

Example:

```
None
{
```

```
  "OrderId": "ORD-INVALID-01",
  "CustomerName": "",
  "Product": "Monitor",
  "Quantity": 0
}
```

This is valid JSON, but it violates our application validation rules.

Encode the message using Base64 before inserting it.

If using PowerShell:

```
None
$payload =
'{"OrderId":"ORD-INVALID-01","CustomerName":"","Produ
ct":"Monitor","Quantity":0}'

$encoded = [Convert]::ToBase64String(
    [System.Text.Encoding]::UTF8.GetBytes($payload)
)

$encoded
```

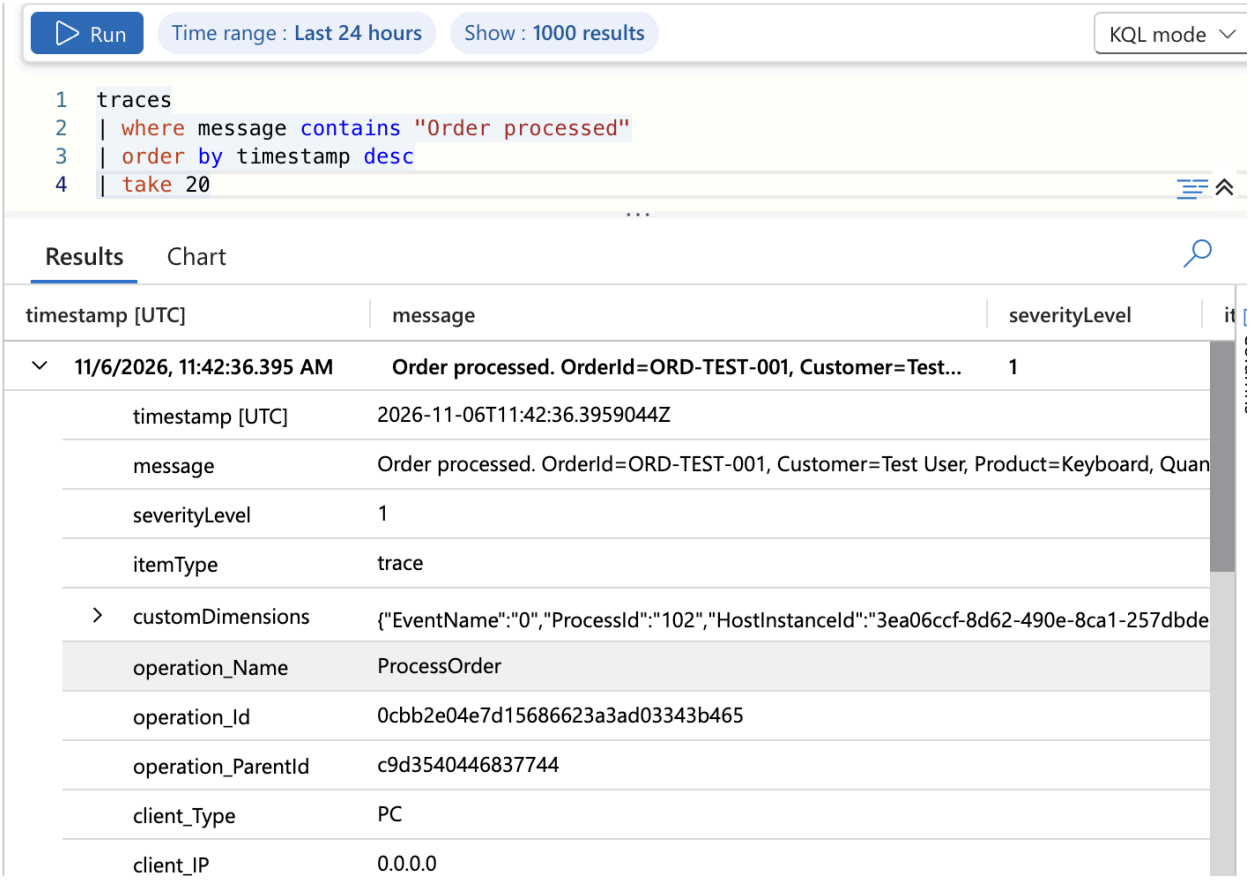
Insert the resulting encoded message.

If your queue management tool provides a Base64 encoding option, use it instead of manually encoding the message. Do not encode it twice.

Step 2 – Observe Function execution

1. Wait for the Function to detect the message.
2. Open Function App → Monitoring → Log Stream.

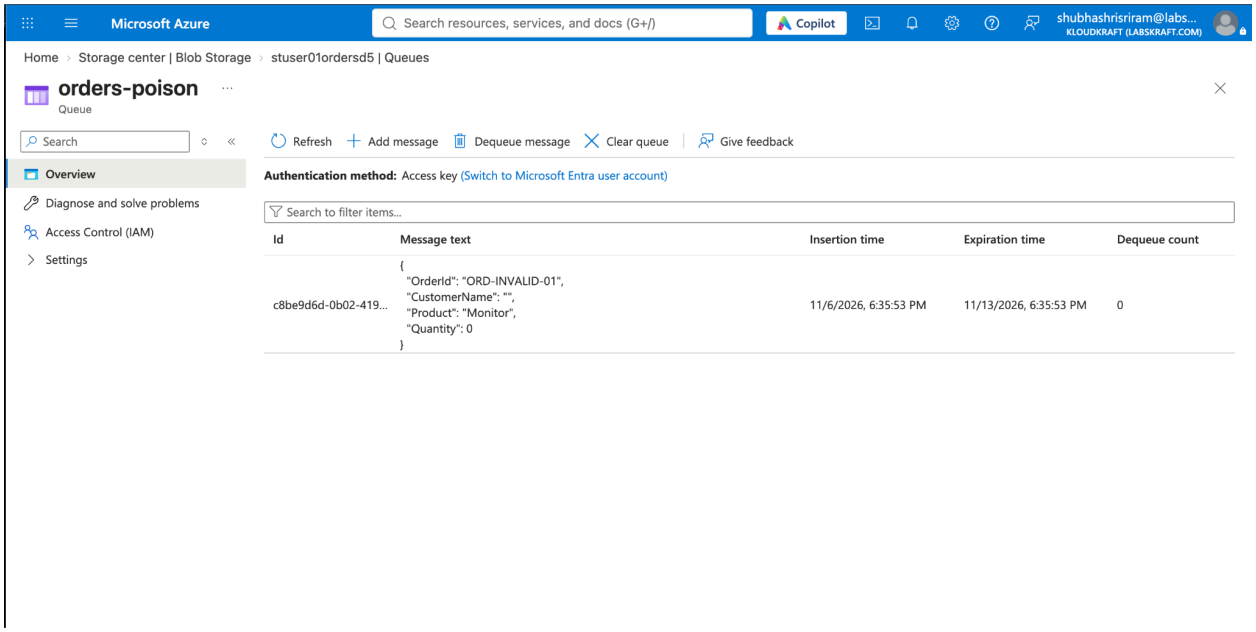
3. Alternatively, open Application Insights → Logs.
4. Observe the order validation failure.
5. Observe repeated processing attempts.



Step 3 – Verify the poison queue

After the configured maximum dequeue attempts, navigate to:

Storage Account → Queues → `orders-poison`.



Verify that the invalid message has been moved.

Expected result

- Invalid order is rejected by the validation logic.
- Function execution fails.
- Retry attempts occur.
- After three unsuccessful attempts, the message moves to `orders-poison`.

Do not manually delete the poison message until evidence has been captured.

Additional debugging exercise – Malformed JSON

The previous example tests business validation, not JSON parsing.

To test the `JsonException` handler separately, insert a Base64-encoded malformed JSON message such as:

```
None
{ invalid-json
```

Observe the JSON parsing exception and confirm that the runtime retries it and eventually moves it to the poison queue.

Engineering understanding

The poison queue prevents a repeatedly failing message from remaining in the normal processing queue indefinitely.

In a production system, support teams investigate the failure, correct the underlying issue and decide whether the message should be repaired and replayed.

---

6. Troubleshooting Guide

Symptom	What to investigate
Function not listed after deployment	Deployment output, generated Function metadata and runtime version.
Function does not start	Log Stream, worker runtime setting and package references.
Queue Trigger cannot connect	OrdersQueue service URI, credential setting and queue RBAC role.
Host fails to start	AzureWebJobsStorage identity settings and Blob Data Owner permission.
Web App sends but Function does not process	Queue Trigger configuration, message encoding and Function logs.
HTTP 403 authorization error	Correct Function identity, role assignment and scope.

Invalid message not moved to poison queue	maxDequeueCount, Function exceptions and Queue extension logs.
Function cannot access Storage	Storage Account availability, endpoint and RBAC propagation.
Function logs missing in Application Insights	Application Insights connection string, telemetry configuration and log filters.
Function receives unreadable message	Producer and consumer Base64 encoding compatibility.

Production troubleshooting approach

When an order is not processed:

1. Confirm the Web App successfully submitted the order.
2. Verify the message was published to the `orders` queue.
3. Check whether the Function App is running.
4. Verify that the `ProcessOrder` Function is listed.
5. Check the Function's Managed Identity and role assignments.
6. Review the Queue Trigger connection configuration.
7. Inspect Log Stream and Application Insights.
8. Check whether the message moved to `orders-poison`.

Operational principle: Identify the failing component before changing code or permissions.

7. Final Validation Checklist

Validation	Expected result
Web App	Running
Production and staging slots	Available

Web App Managed Identity	Enabled
Function App	Running
Functions runtime	Version 4
Function worker	.NET 10 isolated
Hosting plan	Flex Consumption
Function Managed Identity	Enabled
Web App queue permission	Storage Queue Data Message Sender
Function queue permission	Storage Queue Data Contributor
Function host storage permission	Storage Blob Data Owner
Queue Trigger	Configured for `orders`
Identity-based connections	Configured
Valid order	Successfully processed
Invalid order	Rejected by validation
Retry behavior	Repeated attempts observed
Poison queue	Invalid message moved to `orders-poison`
Application Insights	Execution logs available

8. Evidence Collection

Capture:

- Function project structure in VS Code.
- Function project targeting .NET 10.
- Successful dotnet build.
- Function App Overview showing .NET 10 and Flex Consumption.
- Function App Managed Identity enabled.
- Function identity's Storage RBAC role assignments.
- Function code showing the Queue Trigger.
- Function list showing `ProcessOrder`.
- Identity-based Storage application settings.
- Successful order submitted through the Web App.
- Application Insights showing successful processing.
- Invalid order processing attempts.
- `orders-poison` queue showing the failed message.

9. Production Engineering Takeaways

At the end of this lab, participants should understand:

- How event-driven serverless processing works.
- Why Azure Functions is suitable for background workloads.
- The difference between an Azure Function and a Function App.
- How Queue Triggers invoke processing automatically.
- Why the isolated worker model is used.
- How Function host storage differs from trigger-specific storage connections.
- How Managed Identity and Azure RBAC work together.
- How queue messages are serialized and deserialized.
- How retry policies and poison queues handle failures.
- How Application Insights supports production monitoring and troubleshooting.

Real-world corporate scenario

An e-commerce organization receives customer orders through its website.

Instead of processing every operation within the customer-facing request, the application publishes order messages to Azure Queue Storage.

An Azure Function automatically receives and processes those orders in the background.

If a message fails repeatedly, it moves to the poison queue for investigation.

The operations team uses Application Insights to identify failures, trace Order IDs and investigate processing issues.

This architecture separates customer-facing application hosting from background processing and makes each component independently manageable.

10. Day 5 – Complete Architecture

After completing all three labs, the application architecture is:

```
None
            Customer
                |
                v
        ASP.NET Core Web App
           Azure App Service
                |
                | Managed Identity
```

```
        v
Azure Storage Account
        |
        v
   Queue: orders
        |
        | Queue Trigger
        v
Azure Functions
.NET 10 Isolated
        |
        v
Order Validation
Background Processing
        |
        v
Application Insights
Execution Logs
        |
        v
  orders-poison
Failed Messages
```

Final Day 5 outcome

Participants have implemented an end-to-end cloud application workflow involving managed application hosting, secure asynchronous messaging, serverless processing, deployment management, monitoring and failure handling.

11. Cleanup

After completing all three Day 5 labs and capturing the required evidence, remove participant-owned resources.

- Delete the participant's Function App.
- Delete the participant's App Service and deployment slots.
- Delete the participant's App Service Plan.
- Delete the participant's Storage Account.
- Delete participant-owned Application Insights resources if they are separate.

Important: Do not delete the shared Resource Group. Confirm that all participant-owned resources are being removed and that no shared resources are affected.